

Garden Stealth - Java Game Project Report

Executive Summary

Garden Stealth is a 2D stealth-based Java game developed using Java Swing and AWT graphics libraries. The game features a thief character navigating through garden environments, collecting keys and chests while avoiding monsters. The project demonstrates object-oriented programming principles, game development concepts, and Java graphics programming.

Project Type: 2D Stealth Game

Programming Language: Java

Graphics Library: Java Swing/AWT

Development Environment: NetBeans IDE

Build System: Apache Ant

Table of Contents

- [1. Project Overview](#)
- [2. Architecture & Design](#)
- [3. Core Components](#)
- [4. Game Mechanics](#)
- [5. Technical Implementation](#)
- [6. Code Analysis](#)
- [7. Features & Functionality](#)

[8. Project Structure](#)[9. Build & Deployment](#)[10. Conclusion](#)

Project Overview

Game Concept

Garden Stealth is a stealth-based 2D game where players control a thief character navigating through garden environments. The objective is to collect all keys and chests while avoiding detection by monsters, ultimately reaching the exit door to complete each level.

Key Features

- **Multi-level gameplay** with increasing difficulty
- **Stealth mechanics** requiring strategic movement
- **Collectible items** (keys and chests)
- **Enemy AI** with monster patrol patterns
- **Collision detection** system
- **Timer-based scoring**
- **Pause functionality**
- **Sound effects and background music**

Target Platform

- **Operating System:** Cross-platform (Windows, macOS, Linux)
- **Java Version:** Java 8 or higher
- **Memory Requirements:** Minimal (2D graphics)
- **Display:** 768x576 resolution (16:12 aspect ratio)

Architecture & Design

Design Patterns Used

1. MVC (Model-View-Controller) Pattern

- **Model:** Game state, player data, object positions
- **View:** Graphics rendering, UI components
- **Controller:** Key input handling, game loop

2. Object-Oriented Design

- Inheritance hierarchy for game entities
- Polymorphism for different object types
- Encapsulation of game components

3. Component-Based Architecture

- Modular design with separate classes for different functionalities
- Loose coupling between components
- Easy extensibility for new features

Class Hierarchy

```
gardenstealth1 (Main Game Class)
├─ KeyHandler (Input Management)
├─ UserInterface (UI Rendering)
├─ AssetSetter (Object Placement)
├─ Level2manager (Level Management)
├─ sound (Audio Management)
└─ CollisionChecker_1 (Collision Detection)
```

Entity System:

```
Playerclass (Base Entity)
├─ player (Player Character)
└─ Monster (Enemy AI)
```

Object System:

```
SuperObject (Base Object)
├─ OBJ_Key (Collectible Keys)
├─ OBJ_Chest (Collectible Chests)
└─ OBJ_Door (Exit Door)
```

Core Components

1. Main Game Engine (gardenstealth1.java)

Responsibilities:

- Game loop management (60 FPS)
- State management (play, pause, game over)
- Component coordination
- Level progression

Key Methods:

```
public void run() // Main game loop
public void update() // Game state updates
public void paintComponent() // Graphics rendering
public void setupGame() // Initialization
```

Technical Specifications:

- **Screen Resolution:** 768x576 pixels (16x12 tiles, 48px each)
- **World Size:** 50x50 tiles (2400x2400 pixels)
- **Frame Rate:** 60 FPS
- **Tile Size:** 48x48 pixels (16px base × 3x scale)

2. Input Management (KeyHandler.java)

Features:

- WASD/Arrow key movement
- Space bar for game start/restart
- P key for pause functionality
- Key press/release handling

Input Mapping:

Action	Key
Movement	W/A/S/D or Arrow Keys
Action	Space Bar
Pause	P Key

3. Player System (player.java)

Player Attributes:

- Position tracking (worldX, worldY)
- Movement speed (4 pixels/frame)
- Inventory system (keys, chests)
- Collision detection area

4. Collision System (CollisionChecker_1.java)

Collision Types:

- **Tile Collision:** Wall and obstacle detection
- **Object Collision:** Item collection and interaction
- **Entity Collision:** Monster-player contact

Game Mechanics

1. Movement System

- **Grid-based movement** with smooth pixel positioning
- **Collision detection** prevents wall clipping
- **Four-directional movement** (up, down, left, right)
- **Speed control** (4 pixels per frame)

2. Collection Mechanics

- **Key Collection:** Required for door access
- **Chest Collection:** Points and objectives

- **Automatic pickup** on contact
- **Inventory tracking** with visual counters

3. Win/Loss Conditions

Win Conditions:

- Collect all keys in the level
- Collect all chests in the level
- Reach and open the exit door

Loss Conditions:

- Contact with any monster
- Inability to complete objectives (keys/chests lost)

Technical Implementation

1. Graphics System

```
public void paintComponent(Graphics g) { Graphics2D g2 =
    (Graphics2D) g; tileM.draw(g2); // Background tiles
    drawObjects(g2); // Game objects theif.draw(g2); // Player
    character monster.draw(g2); // Enemy entities ui.draw(g2); //
    User interface }
```

Graphics Features:

- **Double buffering** for smooth rendering
- **Sprite-based graphics** with image loading
- **Layered rendering** (background → objects → entities → UI)
- **Coordinate transformation** (world to screen)

2. Game Loop Architecture

```
while (gameThread != null) { update(); // Game logic repaint();  
// Graphics update // Frame rate control  
Thread.sleep(calculatedDelay); }
```

3. Performance Optimization

Optimization Techniques:

- **60 FPS frame limiting**
- **Efficient collision detection**
- **Minimal object creation** during gameplay
- **Smart rendering** (only visible objects)

Code Analysis

1. Code Quality Assessment

Strengths:

- **Modular design** with clear separation of concerns
- **Consistent naming conventions**
- **Comprehensive collision detection**
- **Extensible architecture**

Areas for Improvement:

- **Magic numbers** could be constants
- **Error handling** could be more robust
- **Documentation** could be enhanced
- **Code duplication** in some areas

2. Algorithm Complexity

Operation	Complexity	Description
Collision Detection	$O(n)$	where n = number of objects
Rendering	$O(n)$	where n = number of visible objects
Game Loop	$O(1)$	Constant time per frame
Memory Usage	$O(n)$	Linear with world size

Features & Functionality

1. Core Gameplay Features

Feature	Status	Description
Multi-directional movement	✔ Implemented	WASD/Arrow key controls
Object collection system	✔ Implemented	Keys and chests collection
Collision detection	✔ Implemented	Comprehensive collision system
Enemy AI	✔ Implemented	Basic monster patrol patterns
Level progression	✔ Implemented	2 levels with increasing difficulty
Timer system	✔ Implemented	Real-time game timer
Pause functionality	✔ Implemented	P key to pause/resume
Advanced AI behaviors	✘ Not Implemented	Pathfinding and complex behaviors

Save/load system	✖ Not Implemented	Game state persistence
------------------	-------------------	------------------------

Project Structure

Directory Organization

```
src/
├─ javaapplication/      # Main game classes
│   ├─ gardenstealth1.java # Main game engine
│   ├─ KeyHandler.java    # Input management
│   ├─ UserInterface.java # UI rendering
│   ├─ AssetSetter.java   # Object placement
│   ├─ Level2manager.java # Level management
│   └─ sound.java         # Audio system
├─ entity/               # Game entities
│   ├─ player.java        # Player character
│   ├─ Playerclass.java   # Base entity class
│   ├─ tilemanager.java   # Tile management
│   └─ tile.java          # Tile class
├─ collision/            # Collision system
│   ├─ CollisionChecker_1.java
│   ├─ SuperObject.java   # Base object class
│   ├─ OBJ_Key.java        # Key object
│   ├─ OBJ_Chest.java      # Chest object
│   └─ OBJ_Door.java       # Door object
├─ monster/              # Enemy AI
├─ tiles/                # Tile graphics
├─ sound/                # Audio files
├─ maps/                 # Level maps
└─ other/                 # Additional resources
```

Build & Deployment

Build Process

Prerequisites:

- Java Development Kit (JDK) 8 or higher
- Apache Ant build tool
- NetBeans IDE (recommended)

```
# Clean build ant clean # Compile project ant compile # Build JAR file ant jar # Run project ant run
```

System Requirements

Component	Minimum	Recommended
Operating System	Windows 7+, macOS 10.10+, Linux	Windows 10+, macOS 10.15+, Ubuntu 18.04+
Java Version	JRE 8	JRE 11
Memory	512MB RAM	1GB RAM
Storage	50MB free space	100MB free space
Display	800x600 resolution	1024x768 resolution

Conclusion

Project Summary

Garden Stealth successfully demonstrates the implementation of a complete 2D game using Java Swing and AWT. The project showcases solid object-oriented design principles, effective game loop implementation, and modular architecture that promotes maintainability and extensibility.

Technical Achievements

✓ Successfully Implemented:

- Complete game engine with 60 FPS performance
- Robust collision detection system
- Multi-level gameplay progression
- Comprehensive input handling
- Professional UI with real-time feedback
- Modular, extensible architecture

Future Enhancements

Potential Improvements:

1. **Advanced AI:** Implement pathfinding and more sophisticated monster behaviors
2. **Graphics Enhancement:** Add animations, particle effects, and visual polish
3. **Audio System:** Complete sound effect implementation and audio mixing
4. **Save System:** Add game state persistence and checkpoint system
5. **Level Editor:** Create tools for custom level design
6. **Multiplayer:** Add cooperative or competitive multiplayer modes
7. **Mobile Support:** Port to Android using similar Java-based frameworks

Educational Value

This project serves as an excellent example of:

- **Game Development Fundamentals:** Game loops, collision detection, state management
- **Java Programming:** Swing/AWT graphics, event handling, object-oriented design
- **Software Architecture:** Modular design, separation of concerns, extensibility
- **Project Management:** Code organization, build systems, deployment strategies

Overall Grade: A-

Final Assessment: Garden Stealth represents a well-executed Java game development project that successfully combines technical implementation with engaging gameplay mechanics. The codebase demonstrates professional development practices and provides a solid foundation for future enhancements and learning opportunities.

The project successfully meets its objectives while maintaining high code quality and demonstrating strong software engineering principles. With minor improvements in documentation and error handling, this would be an exemplary game development project.

Report generated on: December 2024

Project Analysis: Garden Stealth Java Game

Total Lines of Code: ~1,500+

Development Time: Estimated 2-3 months